DEPI · SYSTEM ANALYSIS & DESIGN · DELIVERABLE 1 OF 6

Problem Statement, Use Cases, Requirements & Architecture

Section 4.1 — Problem statement and objectives, use case diagram and descriptions, functional and non-functional requirements, software architecture

MediCare — Clinic Management & Appointment System

ASP.NET Core MVC · N-Tier · Microsoft Azure

Team: Mohamed Munir (Team Lead) · Malak Hany · Ashraf Ibrahim · Leqaa Ibrahim · Natalie Bola

November 2026

1. Problem Statement & Objectives

Problem statement

Many small and mid-sized clinics still manage appointments by phone or paper. This causes double bookings, missed appointments, lost records, and no visibility into clinic performance.

Objectives

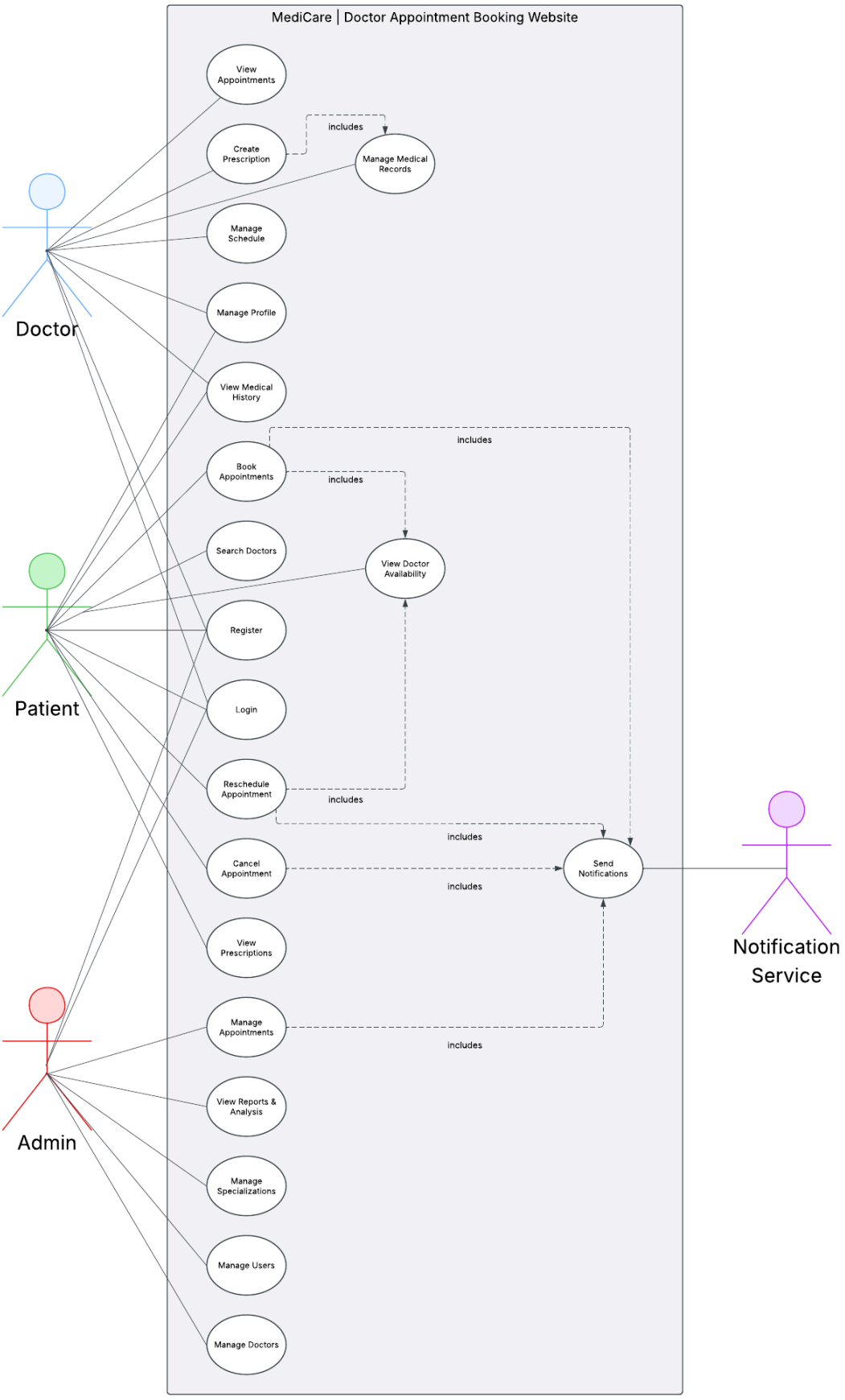
- Provide a single platform for booking, managing, and tracking appointments without conflicts.
- Give doctors a dashboard for schedules, patients, medical records, and prescriptions.
- Keep complete, secure digital medical records per patient.
- Send real-time (SignalR), SMS (Twilio), and email (SendGrid) notifications and reminders.
- Give admins analytics (Chart.js) and management tools.
- Deploy a live, documented system on Microsoft Azure.

Scope

- **In:** multi-role authentication; doctor listing with search/filter; calendar-based booking, cancel, reschedule, conflict detection; doctor dashboard and working hours/vacations; medical records and file uploads; digital prescriptions with print format; notifications; patient profile; admin dashboard; analytics and PDF/Excel export; dark/light mode; Azure deployment with CI/CD.
- **Out (this release):** online payment gateway, video consultations, insurance claims, native mobile app, multi-clinic/multi-branch, integration with external hospital systems.

Purpose from the use case description

A single platform that connects patients, doctors and administrators to simplify managing medical appointments and accessing healthcare services online (see `../02_Use_Case_Diagram_and_Descriptions`).



Use Case Diagram Medicare Description

1. System Overview

MediCare is a doctor appointment booking website designed to simplify the process of managing medical appointments and accessing healthcare services online. The system connects patients, doctors, and

administrators through a single platform, allowing each user to perform specific activities according to their role.

Patients can create accounts, search for doctors, check their availability, book appointments, and manage their medical information. Doctors can manage their schedules, view appointments, create prescriptions, and maintain patients' medical records. Administrators are responsible for managing users, doctors, and medical specializations, as well as viewing reports and analyses.

The system also includes a notification service to support communication related to appointment management. Overall, MediCare aims to make appointment booking more convenient, improve organization, and simplify the management of healthcare-related information.

2. Actors in the System

The use case diagram identifies three primary actors who interact with the MediCare system: Patient, Doctor, and Admin. It also includes a Notification Service that supports the system's notification functionality.

2.1 Patient

The Patient is the primary user of the website. Patients use MediCare to find suitable doctors, schedule medical appointments, and access information related to their healthcare.

The Patient can perform the following activities:

- **Register:** Create a new account by providing the required personal information.
- **Login:** Access the system using existing account credentials.
- **Search Doctors:** Search for doctors according to their needs.
- **View Doctor Availability:** Check the available appointment times for a selected doctor before booking.
- **Book Appointments:** Schedule an appointment with a doctor based on the available time slots.
- **Reschedule Appointment:** Change the date or time of an existing appointment when necessary.
- **Cancel Appointment:** Cancel a previously booked appointment.
- **Manage Appointments:** Access and manage booked appointments, including relevant appointment details and changes.
- **View Prescriptions:** View prescriptions associated with their medical consultations.
- **View Medical History:** Access their recorded medical history to review previous medical information.

These functionalities allow patients to manage their appointments and access relevant medical information through one platform, reducing the need for manual coordination.

2.2 Doctor

The Doctor is a healthcare provider who uses the system to organize appointments, manage availability, and maintain patient-related medical information.

The Doctor can perform the following activities:

- **View Appointments:** Review scheduled appointments to stay informed about upcoming consultations.
- **Manage Schedule:** Set up and update availability to indicate when appointments can be booked.
- **Manage Profile:** View and update professional or personal profile information maintained in the system.
- **Create Prescription:** Create prescriptions for patients when medically appropriate.
- **Manage Medical Records:** Maintain and update patient medical records according to the system's supported functionality.

These functionalities help doctors organize their schedules, review their appointments, and manage medical information more efficiently.

2.3 Admin

The Admin is responsible for overseeing and maintaining the MediCare platform. The administrator manages essential system information and ensures that the platform's users and medical services are organized.

The Admin can perform the following activities:

- **Manage Users:** Manage user accounts and their information within the system.
- **Manage Doctors:** Manage doctor information and the doctors registered on the platform.
- **Manage Specializations:** Add, update, or manage medical specializations available on the website.
- **View Reports and Analysis:** Access reports and analytical information about the system to support monitoring and administrative decision-making.

These functionalities help maintain the organization of the platform, keep its information structured, and support the administration of healthcare services.

2.4 Notification Service

The Notification Service is an external or supporting service represented in the diagram to handle notifications generated by the system.

Its role is to support the sending of notifications related to relevant system activities, particularly appointment management. For example, notifications may inform users about appointment bookings, rescheduling, or cancellations, depending on the notification rules implemented in the system.

This functionality helps keep users informed about important updates and reduces the likelihood of missed or overlooked appointment changes.

3. Main System Functionalities

The use cases can be grouped into the following functional areas:

A. Account Management

Patients can register and log in to access the website's features. Doctors can manage their profiles, while administrators can manage user accounts and doctor information.

B. Doctor Search and Appointment Management

Patients can search for doctors, view their availability, and book appointments. They can also reschedule or cancel existing appointments and manage their bookings. Doctors can manage their schedules and view their appointments to coordinate consultations.

C. Medical Information Management

Patients can view their prescriptions and medical history. Doctors can create prescriptions and manage medical records, supporting the organization of medical information within the platform.

D. Administrative Management

Administrators can manage users, doctors, and medical specializations. They can also access reports and analyses to monitor the system and support administrative decisions.

E. Notifications

The notification service supports communication between the system and its users by sending relevant updates about appointment-related activities.

4. Relationships Between Use Cases

The diagram also illustrates relationships between certain use cases using the UML include relationship.

An include relationship indicates that one use case incorporates the behavior of another use case as part of its execution. For example, if booking an appointment includes viewing doctor availability, the availability-checking functionality is performed as part of the booking process.

In MediCare, these relationships represent dependencies between related operations. They help demonstrate how individual functions work together to complete a larger task, rather than operating entirely independently.

The specific behavior of each relationship should follow the connections shown in the diagram and the functionality defined during system implementation.

5. Overall System Workflow

The general workflow of MediCare begins when a patient registers for an account or logs in to an existing account. The patient can then search for doctors and check their availability before booking an appointment.

After an appointment has been booked, the patient can manage the appointment by viewing its details, rescheduling it, or cancelling it when necessary. The notification service supports the communication of relevant updates.

On the doctor's side, the doctor can manage their schedule and view appointments to organize consultations. The doctor can also create prescriptions and manage medical records as part of the medical information management process.

Meanwhile, the administrator oversees the platform by managing users, doctors, and medical specializations and by accessing reports and analyses.

Through the interaction of these actors and functionalities, MediCare provides an organized environment for managing doctor appointments and supporting the exchange and administration of relevant healthcare information.

6. Conclusion

The MediCare use case diagram provides an overview of the system's functional requirements and illustrates how patients, doctors, administrators, and the notification service interact with the platform.

By separating responsibilities according to user roles, the system supports patient appointment booking, doctor schedule management, medical information handling, administrative operations, and appointment-related notifications.

The diagram serves as a foundation for understanding the system's scope and can guide the subsequent stages of analysis, database design, interface development, and implementation.

Functional Requirements (SRS)

Priorities follow MoSCoW (Must / Should / Could). Requirements are grouped by project week.

Week 1 – Initial Setup and User Management

Scope. N-Tier solution setup, database design, repository and Unit of Work layer, multi-role authentication with role-specific registration, and the doctor listing page with search and filters.

Role	Description in Week 1
Patient	Registers an account, signs in, and browses the doctor listing
Doctor	Registers with professional details, signs in, and appears in the doctor listing
Admin	Pre-created account that signs in with full administrative role access
Visitor	Unauthenticated user who can view the doctor listing and the registration and login pages

N-Tier architecture

ID	Requirement	Priority
FR-ARC-01	The solution shall contain three projects: MediCare.Web (presentation), MediCare.Services (business logic), and MediCare.Data (data access)	Must
FR-ARC-02	MediCare.Web shall contain the MVC controllers, views, and client-side logic, and shall not access the database directly	Must
FR-ARC-03	MediCare.Services shall contain the business rules and shall be the only layer called by controllers	Must
FR-ARC-04	MediCare.Data shall contain the entities, the DbContext, the repositories, and the Unit of Work	Must
FR-ARC-05	Project references shall flow in one direction only: Web to Services to Data	Must
FR-ARC-06	Services and repositories shall be registered in the dependency injection container and injected into controllers and services through their interfaces	Must

Database (schema defined with EF Core and created through migrations)

ID	Requirement	Priority
FR-DB-01	The system shall store doctors with personal information, specialization, consultation fee, profile image, and location	Must
FR-DB-02	The system shall store patients with personal information, medical history, and allergies	Must
FR-DB-03	The system shall store appointments with details, status, and notes, linked to one doctor and one patient	Must
FR-DB-04	The system shall store medical specializations as a separate table, and each doctor shall belong to one specialization	Must
FR-DB-05	The system shall store working hours (doctor availability schedules) linked to a doctor, with day of week, start time, and end time	Must
FR-DB-06	The system shall store medical records (patient medical records and history) linked to a patient and a doctor	Must
FR-DB-07	Relationships, required fields, maximum lengths, and foreign key constraints shall be defined for every table	Must
FR-DB-08	The database shall be created and updated through EF Core migrations, and a seed routine shall load the three roles, the initial Admin account, and a starter list of specializations	Must

Repository and Unit of Work

ID	Requirement	Priority
FR-REP-01	A generic repository shall provide create, read (by ID and list), update, and delete operations for all entities	Must
FR-REP-02	IDoctorRepository shall extend the generic repository with queries for doctors by specialization, location, and availability	Must
FR-REP-03	IPatientRepository shall extend the generic repository with patient-specific queries	Must
FR-REP-04	IAppointmentRepository shall extend the generic repository with appointment queries by doctor, patient, and date	Must
FR-REP-05	IMedicalRecordRepository shall extend the generic repository with queries by patient	Must
FR-REP-06	A Unit of Work shall expose all repositories and a single save operation that commits related changes in one transaction	Must
FR-REP-07	If any part of a grouped operation fails, the Unit of Work shall roll back the entire transaction	Must

Authentication and registration (ASP.NET Identity, roles Patient, Doctor, Admin)

ID	Requirement	Priority
FR-AUTH-01	The system shall define the roles Patient, Doctor, and Admin and assign every user exactly one role	Must
FR-AUTH-02	The system shall provide a patient registration form collecting full name, email, phone number, date of birth, gender, and password, with optional medical history and allergies	Must
FR-AUTH-03	The system shall provide a doctor registration form collecting full name, email, phone number, password, specialization, consultation fee, location, and profile image	Must
FR-AUTH-04	Admin accounts shall not be self-registered; they shall be created by the seed routine	Must
FR-AUTH-05	Registration shall validate required fields, email format, unique email, phone number format, and password strength (minimum length, upper and lower case, digit, and symbol)	Must
FR-AUTH-06	Doctor registration shall also validate that the consultation fee is a positive number and that the profile image is a permitted type (JPG or PNG) within the size limit	Must
FR-AUTH-07	Registration shall create the Identity user, assign the correct role, and create the matching Patient or Doctor profile in a single transaction	Must
FR-AUTH-08	The system shall let registered users sign in with email and password and sign out	Must
FR-AUTH-09	After sign-in, the system shall redirect the user to a landing page appropriate to their role	Must
FR-AUTH-10	The system shall restrict controllers and actions by role, redirect unauthenticated users to the login page, and unauthorized users to an access-denied page	Must

ID	Requirement	Priority
FR-AUTH-11	The system shall lock an account temporarily after repeated failed sign-in attempts	Should
FR-AUTH-12	Passwords shall be stored hashed through ASP.NET Identity and shall never be displayed or logged	Must

Doctor listing

ID	Requirement	Priority
FR-DOC-01	The system shall provide a doctor listing page showing each doctor's name, photo, specialization, consultation fee, and location	Must
FR-DOC-02	The doctor listing shall be visible to visitors, patients, doctors, and admins	Must
FR-DOC-03	The user shall be able to search doctors by name	Must
FR-DOC-04	The user shall be able to filter doctors by specialization, chosen from the specializations table	Must
FR-DOC-05	The user shall be able to filter doctors by location	Must
FR-DOC-06	The user shall be able to filter doctors by availability, based on the doctors' working hours (for example, by day of week)	Must
FR-DOC-07	Search and filters shall be combinable, and results shall show only doctors matching all selected criteria	Must
FR-DOC-08	The user shall be able to clear all filters and return to the full list	Should
FR-DOC-09	When no doctor matches, the page shall show a clear "no results" message	Must
FR-DOC-10	The listing shall be paginated when the number of doctors exceeds one page	Should
FR-DOC-11	Each listing entry shall link to a doctor details page	Should

Out of scope for Week 1: appointment booking, calendar, cancellation, rescheduling and conflict detection; doctor dashboard and working-hours management through the interface; admin approval of doctor registrations; notifications (real-time, SMS, email); medical record entry, file uploads and prescriptions; patient profile page, admin dashboard, analytics and Azure deployment. The appointments, working hours and medical records tables are created in Week 1 as database structure only.

Week 1 deliverables → requirements

Week 1 deliverable	Requirements
Full N-Tier architecture with separation of Presentation, Business Logic, and Data Access	FR-ARC-01 to FR-ARC-06
Multi-role authentication and registration system	FR-AUTH-01 to FR-AUTH-12
Database schema in EF Core with repositories and Unit of Work	FR-DB-01 to FR-DB-08, FR-REP-01 to FR-REP-07
Doctor listing page with search and filter by specialization, location, and availability	FR-DOC-01 to FR-DOC-11

Week 2 – Appointment System and Doctor Dashboard

ID	Requirement	Priority
FR-2.01	The system shall show each doctor's available time slots in an interactive calendar (FullCalendar.js)	Must
FR-2.02	Patients shall book an appointment by selecting a date and time based on the doctor's availability	Must
FR-2.03	Patients shall cancel and reschedule their appointments	Must
FR-2.04	The system shall detect conflicts and prevent double bookings, including simultaneous requests for the same slot	Must
FR-2.05	Appointments shall have the statuses Pending, Confirmed, Rejected, Cancelled, and Completed	Must
FR-2.06	A Factory pattern shall create the different types of appointments and notifications	Must
FR-2.07	Doctors shall view daily and weekly appointments in calendar and list views	Must
FR-2.08	Doctors shall manage their working hours and vacation days	Must
FR-2.09	Doctors shall confirm or reject appointment requests	Must
FR-2.10	Doctors shall quickly access patient information from an appointment	Should
FR-2.11	The system shall notify the doctor instantly (SignalR) when a new appointment is booked	Must
FR-2.12	The system shall notify the patient instantly (SignalR) when an appointment is confirmed or rejected	Must
FR-2.13	The system shall send reminders before scheduled appointments	Should
FR-2.14	The booking system and conflict handling shall be tested comprehensively	Must

Week 3 – Medical Records, Prescriptions, External Notifications and Patient Profile

Medical records management

ID	Requirement
FR-3.1	The system shall allow an authorized doctor to create a medical record for a patient visit
FR-3.2	The system shall allow the doctor to record the patient's diagnosis, symptoms, and treatment notes for each visit
FR-3.3	The system shall allow authorized users to upload and attach medical documents, including test results, X-rays, and other relevant files
FR-3.4	The system shall allow authorized users to view a patient's medical history and previous visit records
FR-3.5	The system shall associate each medical record with the relevant patient and visit
FR-3.6	The system shall restrict access to medical records and attached documents according to user roles and permissions

Prescription system

ID	Requirement
FR-3.7	The system shall allow an authorized doctor to create a digital prescription for a patient
FR-3.8	The system shall allow the doctor to add medication details, dosage, frequency, and usage instructions to a prescription
FR-3.9	The system shall save each prescription and associate it with the relevant patient and visit
FR-3.10	The system shall allow patients to view their prescription history
FR-3.11	The system shall provide a print-friendly prescription format and allow the prescription to be printed

External notifications

ID	Requirement
FR-3.12	The system shall send SMS appointment reminders through Twilio when the integration is configured
FR-3.13	The system shall send email notifications through SendGrid when the integration is configured
FR-3.14	The system shall schedule an automated reminder 24 hours before a scheduled appointment
FR-3.15	The system shall allow users to manage their notification preferences
FR-3.16	The system shall notify patients when an appointment is confirmed or rejected, using the notification channels enabled for that patient

Patient profile

ID	Requirement
FR-3.17	The system shall allow patients to view their upcoming appointments and appointment history
FR-3.18	The system shall allow patients to access their medical records and prescriptions, subject to authorization rules
FR-3.19	The system shall allow patients to update their personal information and medical history
FR-3.20	The system shall allow patients to download medical documents that they are authorized to access

Week 4 – Admin Dashboard, Analytics and Deployment

ID	Requirement	Priority
FR-4.01	Admins shall approve doctor registrations and update doctor profiles	Must
FR-4.02	Admins shall manage patients and their access	Must
FR-4.03	Admins shall manage specializations and clinic settings	Must
FR-4.04	Admins shall view and manage all appointments	Must
FR-4.05	The system shall display monthly appointment statistics (Chart.js)	Must
FR-4.06	The system shall display the most popular doctors and specializations	Must
FR-4.07	The system shall display patient demographics and trends	Must

ID	Requirement	Priority
FR-4.08	The system shall provide revenue reports and financial summaries, where revenue = the consultation fee of each Completed appointment (no online payments in this release)	Must
FR-4.09	Reports shall be exportable to PDF and Excel	Should
FR-4.10	The UI shall be responsive (Bootstrap 5) with intuitive navigation	Must
FR-4.11	The UI shall include accessibility improvements	Must
FR-4.12	The UI shall offer a dark/light mode toggle	Could
FR-4.13	The application shall be deployed to Azure App Service with Azure SQL Database	Must
FR-4.14	Real-time features shall run on Azure SignalR Service	Must
FR-4.15	A CI/CD pipeline shall be set up with Azure DevOps or GitHub Actions	Should
FR-4.16	The project shall include documentation: architecture explanation with diagrams, API documentation, setup guide, and user manual	Must

Non-Functional Requirements (SRS)

Category	Requirements
Performance	NFR-1: Main pages load in under 2 seconds under normal load (about 50 concurrent users). NFR-2: Appointment conflict check completes in under 1 second. NFR-3: Real-time notifications arrive within 3 seconds.
Security and privacy	NFR-4: Passwords are hashed using ASP.NET Identity; HTTPS is enforced. NFR-5: Role-based and resource-based authorization: patients only see their own data; doctors only see their patients. NFR-6: Protection against SQL injection (EF Core parameterization), XSS, and CSRF (anti-forgery tokens). NFR-7: Uploaded files are validated by type and size and stored securely. NFR-8: Secrets (API keys, connection strings) are stored in configuration/Key Vault, never in source control. Medical records, prescriptions and attached documents are visible only to authorized users; one patient's information is never exposed to another patient. Accounts lock temporarily after repeated failed sign-in attempts; passwords are never displayed or logged. Compliance by design with health-data regulations (HIPAA, GDPR, local laws): encryption, role-based access, a basic audit log of key actions (sign-in, record access, appointment status changes), consent, and a privacy policy; sensitive details are kept out of SMS and email messages. Upload file types and size limits, and notification retry behavior, are agreed during requirements gathering.
Usability and accessibility	NFR-9: A new patient can book an appointment in under 2 minutes without training. NFR-10: The UI is responsive on mobile, tablet, and desktop, and follows basic accessibility guidelines (labels, contrast, keyboard navigation). NFR-11: Clear error messages and form validation.
Reliability and availability	NFR-12: Target uptime of at least 99.5% during the evaluation period. NFR-13: Related changes are saved in a single transaction (Unit of Work) to avoid inconsistent data. NFR-14: Notification failures are logged and retried without breaking the booking flow.
Maintainability and scalability	NFR-15: Strict N-Tier separation (Web / Services / Data) with Repository, Unit of Work, and Dependency Injection. NFR-16: Code follows consistent naming conventions and is reviewed through pull requests. NFR-17: Business logic is unit-testable through interfaces. NFR-18: The design allows adding new notification channels or appointment types without major changes. NFR- 22: Code follows SOLID principles and public APIs carry XML documentation. NFR-23: Business logic is covered by unit tests and the repository layer by integration tests; critical paths (sign-in, booking, prescription) get basic UI tests. NFR-24: Errors are handled consistently and logged centrally (level, user/request context), with no sensitive data in logs.
Compatibility and portability	NFR-19: Works on current versions of Chrome, Edge, Firefox, and Safari. NFR-20: Database changes are managed through EF Core migrations.
Documentation	NFR-21: Architecture explanation with diagrams, API documentation, setup guide, and user manual are provided in the repository.

3. Functional & Non-Functional Requirements — Requirements Traceability Matrix

Functional requirements

ID	Requirement	Actor	Design components	Test (RTM)	Priority	Scope note
FR-01	Allow a new user to create an account by submitting required registration data	Patient / Doctor	UI; Authentication; User Management	TC-01	High	Core — confirm roles
FR-02	Allow registered users to log in with valid credentials	Patient / Doctor / Admin	UI; Authentication; User Mgmt	TC-02	High	Core
FR-03	Role-appropriate access for patients, doctors, administrators	All	Authentication; User Mgmt	TC-03	High	Core
FR-04	View and update permitted profile information	Patient / Doctor	User Mgmt	TC-04	Medium	Core — confirm fields
FR-05	Browse/search available doctors and view details	Patient	Doctor Mgmt; UI	TC-05	High	Core
FR-06	Display doctor availability and available slots	Patient	Doctor Mgmt; Appointment Mgmt	TC-06	High	Core
FR-07	Request an appointment with a selected doctor and time slot	Patient	Appointment Mgmt; Database	TC-07	High	Core
FR-08	Prevent two patients booking the same doctor/time slot	Patient / System	Appointment Mgmt; Database	TC-08	High	Core
FR-09	Track appointment status: pending, confirmed, rejected, cancelled, completed	Patient / Doctor	Appointment Mgmt	TC-09	High	Core — confirm Completed state
FR-10	Doctors review appointment requests and accept or reject	Doctor	Appointment Mgmt; Doctor Mgmt	TC-10	High	Core
FR-11	Cancellation per the project's cancellation rules	Patient / Doctor (as approved)	Appointment Mgmt	TC-11	Medium	Core — confirm cancellation rules
FR-12	Authorized users view relevant medical records	Patient / Doctor	Medical Records; Authentication	TC-12	High	Core
FR-13	Authorized clinical users create/update medical records after a consultation	Doctor	Medical Records; Database	TC-13	High	Core — confirm record fields
FR-14	Prescription creation or viewing (if in approved scope)	Doctor / Patient	Prescription System; Medical Records	TC-14	Medium	**Confirm scope**
FR-15	Admins manage user accounts and doctor information	Admin	Admin Mgmt; User Mgmt; Doctor Mgmt	TC-15	High	Core — confirm admin actions
FR-16	Admins review system/activity reports (if in scope)	Admin	Admin Mgmt; Reporting	TC-16	Medium	**Confirm scope**
FR-17	Send appointment-related notifications (if enabled)	System / Patient / Doctor	Notification Service; Appointment Mgmt	TC-17	Medium	**Confirm channel and scope**
FR-18	Waitlist when no slot is available (if approved for this release)	Patient	Appointment Mgmt; Waitlist	TC-18	Medium	**Confirm scope** — omitted from latest State Diagram

ID	Requirement	Actor	Design components	Test (RTM)	Priority	Scope note
FR-19	Record allergy/medication info and alert authorized clinical users (if approved)	Patient / Doctor	Patient Profile; Medical Records; Allergy/Medication Alerts	TC-19	High	**Confirm scope** — omitted from latest State Diagram
FR-20	Patients submit doctor ratings/reviews after an eligible appointment (if in scope)	Patient	Doctor Mgmt; Reviews & Ratings	TC-20	Low	**Confirm scope**

RTM test-case descriptions

Test	Verifies
TC-01	Valid data creates account; invalid/missing data shows validation
TC-02	Valid credentials open the correct dashboard; invalid rejected
TC-03	Each role can access permitted features and not restricted ones
TC-04	Profile updates saved; invalid data rejected
TC-05	Search/browse shows correct doctor details
TC-06	Only available slots are offered
TC-07	Booking request stored with correct patient, doctor, time
TC-08	Second conflicting booking for the same slot is rejected
TC-09	Each supported status transition shows the right status
TC-10	Accept/reject updates status for the patient
TC-11	Cancel an eligible appointment; status and slot availability updated
TC-12	Authorized users see permitted records; unauthorized cannot
TC-13	Record update persists with correct patient association and access restrictions
TC-14	Prescription create/view with correct patient association and access
TC-15	Create/update/disable test account or update doctor info
TC-16	Report values match stored test data
TC-17	Booking/status change generates the configured notification
TC-18	Join waitlist for a full slot; request stored and manageable
TC-19	Add test allergy/medication; clinical users see alert
TC-20	Review for an eligible completed appointment validated and stored

Non-functional requirements

ID	Requirement	Components	Test	Priority	Note
NFR-01	Protect personal and medical data using role-based authorization	Authentication; all protected components	TN-01: unauthorized access to another user's medical data is denied	High	Core quality requirement
NFR-02	Validate input and display understandable error messages	UI; Authentication; Appointment Mgmt	TN-02: invalid/missing/malformed values give safe feedback	High	Core
NFR-03	Maintain data consistency for appointments and medical records	Appointment Mgmt; Medical Records; Database	TN-03: competing bookings/updates produce no conflicting or lost data	High	Core
NFR-04	Usable interface on common desktop and mobile screen sizes	UI (Front-End)	TN-04: main screens at desktop, tablet, mobile	Medium	Core

ID	Requirement	Components	Test	Priority	Note
NFR-05	Reasonable response time and availability for core workflows, targets agreed by the team	Front-End; Backend/API; Database	TN-05: measure response time, record uptime vs. targets	Medium	**Set measurable targets**
NFR-06	Audit trail for sensitive record changes (if required)	Medical Records; Admin Mgmt; Database	TN-06: verify who changed a record and when	Medium	**Confirm scope**

MediCare – Software Architecture & Technology Stack

Clinic Management & Appointment System

1. Software Architecture

1.1 Overview

MediCare is a full-featured healthcare platform that connects three user groups: **Patients**, who search for doctors, book appointments and view their medical history; **Doctors**, who manage their schedules and appointments and write prescriptions; and **Admins**, who manage the whole system and view reports. The chosen architecture is **N-Tier**, with **ASP.NET Core MVC** in the presentation layer and a clear separation between presentation, business logic and data access.

1.2 Architectural Style: N-Tier with MVC

The system is split into three main layers. Each layer is its own project in the solution, and dependencies point in one direction only: from the top layer down.

- **MediCare.Web** depends on **MediCare.Services**, which depends on **MediCare.Data**.
- A lower layer must never know about the layer above it (for example, the Data layer knows nothing about the Controllers).
- Layers communicate through **interfaces**, not concrete implementations.

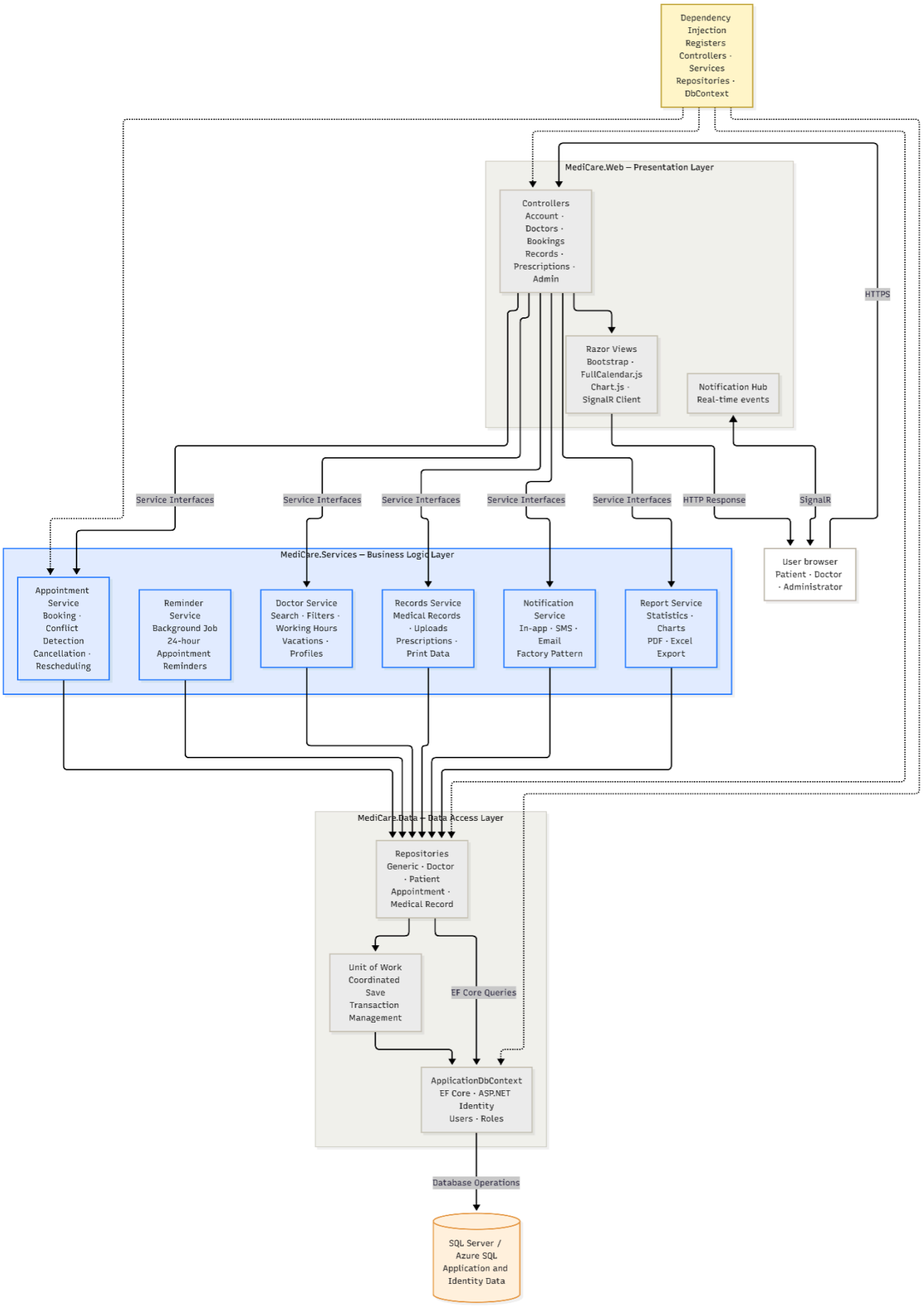
1.3 Layers and Their Components

Layer	Responsibility	Main components
MediCare.Web (Presentation Layer)	Receives user requests, renders pages, performs first-level input validation, and provides real-time communication with the browser	Controllers (Account, Doctors, Appointments, MedicalRecords, Prescriptions, Admin), Razor Views, ViewModels, Areas (Patient / Doctor / Admin), SignalR Hubs (NotificationHub), client JS (FullCalendar.js, Chart.js), Filters and Middleware
MediCare.Services (Business Logic Layer)	Enforces business rules: preventing appointment conflicts, the appointment lifecycle, prescription creation, notifications, and reports	Service interfaces and implementations (IAppointmentService, IDoctorService, IPatientService, IMedicalRecordService, IPrescriptionService, INotificationService, IReportService), DTOs, AutoMapper profiles, validators, factories, background jobs (reminders)
MediCare.Data (Data Access Layer)	Talks to the database and hides EF Core details from the rest of the system	ApplicationDbContext, entities, Fluent API configurations, migrations, generic repository, specific repositories (IDoctorRepository, IPatientRepository, IAppointmentRepository, IMedicalRecordRepository), Unit of Work, seed data

Core entities: Doctors, Patients, Specializations, Appointments, WorkingHours, MedicalRecords, Prescriptions (with PrescriptionItems), plus the ASP.NET Identity tables (Users, Roles).

1.4 Architecture Diagram (Figure 1)

Figure 1 presents the three application layers, their principal components, the dependency direction, the database boundary, and the cross-cutting Dependency Injection and SignalR connections.



Logical N-Tier architecture · three application layers · one database

Figure 1: MediCare N-Tier architecture. The Web layer calls service interfaces; Services use repositories and Unit of Work; the Data layer accesses SQL Server through EF Core. Dependency Injection registers components across the solution, while SignalR supports real-time browser communication.

1.5 Request Flow

Example: **a patient books an appointment**

1. The patient picks a slot in FullCalendar.js, which sends an AJAX/POST request to AppointmentsController.
2. The controller validates the ModelState and the user's role (Patient), then calls IAppointmentService.BookAsync(dto).
3. The service applies the business rules: the slot is inside the doctor's working hours, the doctor is not on leave, and there is no conflicting appointment (conflict detection).
4. The service uses AppointmentFactory to create the right appointment type (consultation, examination, or follow-up).
5. The service calls IUnitOfWork.Appointments.AddAsync(...) followed by CompleteAsync(), saving all changes in a single transaction.
6. After saving, NotificationFactory prepares the notifications: SignalR to the doctor in real time, plus Email/SMS according to preferences.
7. The controller returns the result (JSON or a redirect) and the view updates.

1.6 Design Patterns

Pattern	Where	Purpose and benefit
Repository	MediCare.Data	Hides EF Core queries behind a clear interface (IDoctorRepository, etc.), reduces duplicated code, and makes testing with mocks easy. A generic repository covers common CRUD; specific repositories cover special queries (doctor search by specialization, location and availability; a day's appointments; patient history).
Unit of Work	MediCare.Data	Groups several related operations into one transaction (for example: book an appointment, update availability, log a notification) so they all succeed or all fail, leaving no inconsistent data.
Factory	MediCare.Services	AppointmentFactory creates appointment types (Consultation / Follow-up / Emergency) with different rules, and NotificationFactory creates the right notification (SignalR / SMS / Email) by channel and type. New types can be added without changing existing code (Open/Closed).
Dependency Injection	All layers	Repositories and services are registered in Program.cs and injected into controllers through constructors, giving loose coupling and easy unit testing.
MVC	MediCare.Web	Separates Model, View and Controller inside the presentation layer.
DTO / ViewModel	Web + Services	Entities are never passed directly to views, which protects the domain model and prevents over-posting.

1.7 Cross-Cutting Concerns

Concern	Implementation
Authentication and authorization	ASP.NET Core Identity with three roles (Patient, Doctor, Admin), [Authorize(Roles=...)] and policy-based authorization. A new doctor needs admin approval before activation.
Validation	Data Annotations plus FluentValidation in the services, and client-side validation in the views.
Logging and monitoring	ILogger with Serilog, and Azure Application Insights in production.
Error handling	Global exception-handling middleware and custom error pages.
Real-time communication	A SignalR hub sends instant notifications (new booking, appointment confirmed or rejected, reminders).
Background processing	Hosted Service / Hangfire to schedule the reminder 24 hours before an appointment.
Caching	IMemoryCache for stable data (specializations, clinic settings).
Configuration and secrets	appsettings plus Azure Key Vault for Twilio and SendGrid keys and connection strings.

Concern	Implementation
Auditing	Records who changed a medical record and when, which matters for sensitive medical data.

1.8 Quality Attributes

Attribute	How the architecture achieves it
Maintainability	Separation of concerns: changing one layer does not break the others.
Scalability	A stateless web layer scales horizontally on Azure App Service, and Azure SignalR Service offloads real-time connections.
Testability	Interfaces and DI allow unit tests of services with mock repositories.
Security	Identity, HTTPS, anti-forgery tokens, SQL injection protection through EF Core, protected medical files, and role-based permissions.
Data consistency	Unit of Work, transactions, and a unique constraint on (Doctor + DateTime) to prevent double booking.
Performance	Async/await, pagination, deliberate eager/lazy loading, and indexes on frequently searched fields.
Availability and reliability	Azure SQL with backups, and retry policies for external integrations (Twilio / SendGrid).
Usability and accessibility	Responsive Bootstrap 5, dark/light mode, and ARIA support.

1.9 Why N-Tier?

- **Right size for the project:** a monolith built by a small team in four weeks. N-Tier gives strong structure without operational complexity.
- **Matches the project requirements:** the brief itself asks for Presentation / Business / Data layers with Repository and Unit of Work.
- **Simpler transactions:** one database and one transaction keep bookings and medical records consistent, instead of the distributed-consistency problems of microservices.
- **Lower cost and operations:** a single deployment on Azure App Service instead of managing several services, an API gateway and a message broker.
- **Room to evolve:** services are split by domain (Appointments, Records, Notifications), so a part can later be extracted as a microservice if the system grows.
- **Easy testing and maintenance** thanks to the clear boundaries between layers.

Note: microservices were ruled out because they add complexity (service discovery, distributed transactions, heavier DevOps) that a single-clinic system does not justify.